Interface summary

17 interfaces in geometry.ts, color.ts, shape.ts, window.ts, view.ts. Generated 2026-11-05 by npm run summary. Inherited members are listed under the interface that declares them.

Extends **InkStroke** extends Shape · **ShapeLocator** extends ShapeLocatorQueries · **View** extends FrameHandler

GEOMETRY.TS

Point

A location in canonical coordinates.

```
readonly x: number
readonly y: number
```

Rectangle

An axis-aligned rectangle covering the half-open region [x, x + width) × [y, y + height).

```
readonly x: number
readonly y: number
readonly width: number
readonly height: number
```

RectangleOps

Operations on rectangles, following Rectangle's half-open semantics.

```
contains(r: Rectangle, p: Point): boolean
```
Whether p lies in r.

```
intersects(a: Rectangle, b: Rectangle): boolean
```
Whether a and b overlap.

Viewport

A 2D similarity transform (uniform scale, rotation, translation) from canonical coordinates to view coordinates.

```
readonly scale: number
```
View units per canonical unit.

```
readonly rotation: number
```
Radians.

```
readonly tx: number
readonly ty: number
```

COLOR.TS

Color

An sRGB color with straight (not premultiplied) alpha.

```
readonly r: number
readonly g: number
readonly b: number
readonly a: number
```

SHAPE.TS

Shape

A region of the plane, such as an ink stroke.

```
readonly kind: string
```
Identifies the shape's type, so that code such as a view can tell which kind of shape it has.

```
boundingBox(): Rectangle
```
The exact extents of the shape: x and y are its minimum x and y, and x + width and y + height are its maximum x and y.

```
intersects(r: Rectangle): boolean
```
Whether at least one point lies both in this shape and in r.

```
contains(p: Point): boolean
```
Whether p lies in the shape.

InkStroke extends Shape

A stroke of digital ink: a polyline drawn with a round pen.

```
readonly kind: "ink"
readonly points: readonly Point[]
```
The polyline's vertices, in drawing order.

```
readonly thickness: number
```
Pen diameter, in canonical units.

```
readonly color: Color
```

ShapeCollection<S extends Shape>

A collection of shapes, kept in z-order.

```
add(shape: S): ShapeId
```
Adds shape and returns its new ID.

```
remove(id: ShapeId): boolean
```
Removes the shape with this ID.

```
removeMultiple(ids: readonly ShapeId[]): number
```
Removes every listed ID that is present. ids may contain duplicates and IDs that aren't present; both are ignored.

```
get(id: ShapeId): S | undefined
```
The shape with this ID, or undefined if no such shape exists.

```
readonly size: number
```
The number of shapes in the collection.

```
forEach(visit: (id: ShapeId, shape: S) =>
  void): void
```
Calls visit for each shape, bottom to top in z-order. visit must not modify the collection.

```
compareZ(a: ShapeId, b: ShapeId): number
```
Compares the z-order of two shapes.

ShapeLocatorQueries

Finds shapes by location.

```
inRect(r: Rectangle, out: ShapeId[]): number
```
Finds the shapes that intersect r.

```
atPoint(p: Point, out: ShapeId[]): number
```
Finds the shapes that contain p.

ShapeLocator extends ShapeLocatorQueries

A spatial index that finds shapes by location, such as a quadtree.

```
add(id: ShapeId, shape: Shape): void
```
Indexes shape under id.

```
remove(id: ShapeId): boolean
```
Removes id from the index.

```
removeMultiple(ids: readonly ShapeId[]): number
```
Removes every listed ID that is present. ids may contain duplicates and IDs that aren't present; both are ignored.

WINDOW.TS

FrameHandler

Something that does its work during a display frame.

```
frame(timeMs: number): void
```
timeMs is the frame's timestamp in milliseconds, on the window's monotonic clock.

FrameScheduler

Calls handlers before the window's next display refresh.

```
requestFrame(handler: FrameHandler): void
```
Asks for one call to handler.frame() before the next display refresh.

VIEW.TS

Layer<T>

One layer of a scene: items in z-order, found by location, with a way to learn when they change.

```
get(id: ItemId): T | undefined
```
The item with this ID, or undefined if no such item is present.

```
readonly size: number
```
The number of items in the layer.

```
forEach(visit: (id: ItemId, item: T) => void):
  void
```
Calls visit for each item, bottom to top in z-order. visit must not change the source.

```
compareZ(a: ItemId, b: ItemId): number
```
Compares the z-order of two items.

```
inRect(r: Rectangle, out: ItemId[]): number
```
Finds the items whose regions intersect r.

```
atPoint(p: Point, out: ItemId[]): number
```
Finds the items whose regions contain p.

```
subscribe(listener: () => void): () => void
```
Calls listener after each change, until the returned function is called.

Scene<T>

What views draw: layers in canonical coordinates, the same for every view, with no viewport. layers[0] is the bottom layer.

```
readonly layers: readonly Layer<T>[]
```

ItemRenderer<T, Surface>

Draws items onto a platform's drawing surface, such as a CanvasRenderingContext2D, a CGContext or an SVG builder.

```
draw(item: T, surface: Surface): void
```
Draws item in canonical coordinates.

View<T> extends FrameHandler

Shows a scene on screen through a viewport, and stays current.

```
readonly scene: Scene<T>
viewport: Viewport
```
Setting this redraws the view at the next frame and calls its viewport subscribers.

```
subscribeViewport(listener: (viewport:
  Viewport) => void): () => void
```
Calls listener with the new viewport after each change, until the returned function is called.

```
interactive: boolean
```
When false, the view ignores all input, gestures included (for example, a minimap that is only for display).

Exporter<T, Out>

Produces a document (SVG, PDF and so on) from a scene, once, on request, drawing with an ItemRenderer for its format.

```
export(scene: Scene<T>, viewport: Viewport,
  width: number, height: number): Out
```